Name: Khushi Singh
Class: D20C Roll No.: 34

Experiment No. 7

Aim: To implement the blockchain platform Ganache and deploy a Solidity smart contract for the pharmaceutical supply chain using MetaMask and Remix IDE.

Mini Project: Blockchain-Based Supply Chain Management System for pharmaceuticals.

Course Outcomes (CO):

Tasks Performed

1. Installed Ganache and started a local blockchain (http://127.0.0.1:7545, chain ID 1337).
2. Connected a Ganache-funded account to MetaMask through a custom network.
3. Connected Remix IDE to MetaMask (Browser Extension → MetaMask).
4. Wrote the Solidity contract `PharmaSupplyChain.sol` based on the mini project.
5. Compiled it (solc 0.8.19, EVM paris) and deployed it through MetaMask on Ganache.
6. Checked the transaction details on the Ganache chain (blocks 0–17).
7. Interacted with the contract through the full batch lifecycle, plus tamper and access-control tests.

Theory

1. What is Ganache?

Ganache is a personal Ethereum blockchain for development and testing. It lets developers deploy contracts, build DApps and run tests in a safe, deterministic local environment. Ganache provides:

- Instant mining of blocks (one block per transaction)
- Pre-funded test accounts (10 accounts × 1000 ETH in this run)
- Transaction and block history over a JSON-RPC endpoint
- A deterministic wallet, so the same addresses and keys appear on every start

2. MetaMask and Remix IDE

Ganache is part of the Truffle Suite and is commonly used with **MetaMask** (a browser wallet that signs transactions) and **Remix IDE** (a browser IDE that compiles and deploys Solidity). MetaMask is pointed at Ganache through a custom network, and Remix sends its transactions through MetaMask using the injected provider.

3. Mini-Project Contract: PharmaSupplyChain

`PharmaSupplyChain.sol` is a single-contract version of the BioToken architecture. Each drug batch is a token with three roles and a fixed lifecycle:

Role	Allowed actions
MANUFACTURER	`mint` a batch with its fingerprint, `markInTransit` to a courier
COURIER	holds the batch while it is in transit
LAB	`confirmReceipt`, `verifyProof`, `consumeToken`

Lifecycle: MINTED → IN_TRANSIT → RECEIVED → VERIFIED → CONSUMED, or REJECTED if verification fails.

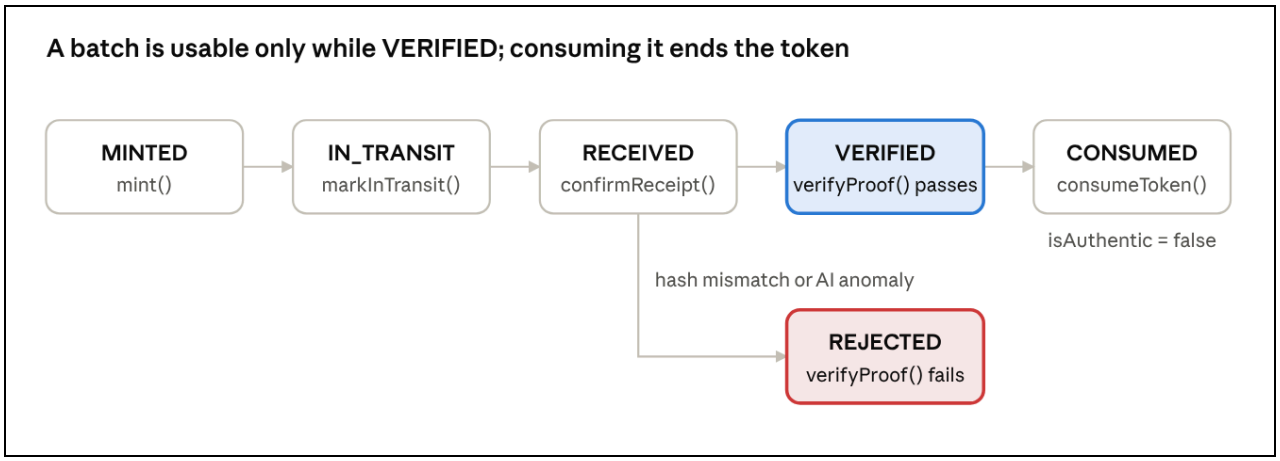


Fig. Batch lifecycle – 6 states, one contract function per transition

Steps to Connect Ganache with MetaMask and Remix IDE

1. **Install and start Ganache.**
   - `npm install --save-dev ganache solc@0.8.19 ethers@6`
   - `npx ganache --chain.chainId 1337 --wallet.deterministic -p 7545 --database.dbPath ./ganache-data`
   - Ganache listens on http://127.0.0.1:7545 with 10 pre-funded accounts of 1000 ETH each.
2. **Connect Ganache to MetaMask.**
   - MetaMask → Add a custom network: RPC URL `http://127.0.0.1:7545`, Chain ID `1337`, symbol `ETH`.
   - Switch to that network and select the MetaMask account (Account 4, `0x0950cb99CC73218Eba72712e268145F581fd87c6`).
   - Fund it from Ganache: 100 ETH sent from Ganache Account 0 (block 11). Importing a Ganache private key into MetaMask works as well.
3. **Connect Remix IDE to MetaMask.**
   - Open https://remix.ethereum.org → Deploy & Run Transactions → Environment **Browser Extension → MetaMask**.
   - Approve the connection in MetaMask. Remix then shows the account, 100 ETH and network (1337).
4. **Create the contract.** In the File Explorer, add `PharmaSupplyChain.sol` (Code section).
5. **Compile and deploy.**
   - Solidity Compiler: version `0.8.19+commit.7dd6d404`, Advanced Configurations → EVM version `paris`, optimization off.
   - Click **Compile**, then **Deploy** in Deploy & Run, and approve the "Deploy a contract" request in MetaMask.
6. **Check the transaction details on Ganache.** Query each block, transaction, gas and balance over JSON-RPC.
7. **Interact with the contract.** Call `mint` → `markInTransit` → `confirmReceipt` → `verifyProof` → `consumeToken` from Remix, confirming each in MetaMask. Read `getBatch` / `isAuthentic` between steps.

Code

PharmaSupplyChain.sol (solc 0.8.19, EVM paris, compiled with no warnings or errors):

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;

/// @title PharmaSupplyChain
/// @notice Single-contract version of the BioToken architecture for a
///         pharmaceutical supply chain. Each batch is a token with a lifecycle
///         MINTED -> IN_TRANSIT -> RECEIVED -> VERIFIED -> CONSUMED
///         (or REJECTED if the lab's fingerprint does not match).
///         The Groth16 ZK proof of the paper is replaced by a fingerprint
///         commitment: keccak256(10 HPLC peaks, secret salt).
contract PharmaSupplyChain {
    // ------------------------------------------------------------------ roles
    bytes32 public constant ADMIN_ROLE        = keccak256("ADMIN");
    bytes32 public constant MANUFACTURER_ROLE = keccak256("MANUFACTURER");
    bytes32 public constant COURIER_ROLE      = keccak256("COURIER");
    bytes32 public constant LAB_ROLE          = keccak256("LAB");

    mapping(bytes32 => mapping(address => bool)) private roles;

    // ------------------------------------------------------------------ lifecycle
    enum Status { MINTED, IN_TRANSIT, RECEIVED, VERIFIED, CONSUMED, REJECTED }

    struct Batch {
        uint256 tokenId;
        string  drugName;
        string  batchNumber;
        uint256 quantity;
        uint256 expiryDate;      // unix timestamp
        bytes32 fingerprint;     // commitment to HPLC peaks + salt
        address manufacturer;
        address courier;
        address lab;
        address currentHolder;
        Status  status;
        uint256 mintedAt;
    }

    struct HistoryEntry {
        Status  status;
        address actor;
        uint256 timestamp;
    }

    uint256 public totalBatches;
    mapping(uint256 => Batch) private batches;
    mapping(uint256 => HistoryEntry[]) private history;

    // ------------------------------------------------------------------ events
    event RoleGranted(bytes32 indexed role, address indexed account, address indexed by);
    event RoleRevoked(bytes32 indexed role, address indexed account, address indexed by);
    event BatchMinted(uint256 indexed tokenId, string drugName, string batchNumber, uint256
quantity, address indexed manufacturer, bytes32 fingerprint);
    event BatchInTransit(uint256 indexed tokenId, address indexed courier, uint256 timestamp);
    event BatchReceived(uint256 indexed tokenId, address indexed lab, uint256 timestamp);
    event BatchVerified(uint256 indexed tokenId, address indexed lab, uint256 timestamp);
    event BatchRejected(uint256 indexed tokenId, address indexed lab, string reason, uint256
timestamp);
    event BatchConsumed(uint256 indexed tokenId, address indexed lab, uint256 timestamp);

    // ------------------------------------------------------------------ modifiers
    modifier onlyRole(bytes32 role, string memory err) {
        require(roles[role][msg.sender], err);
        _;
    }

    modifier exists(uint256 tokenId) {
```

```
        require(tokenId > 0 && tokenId <= totalBatches, "Batch does not exist");
        _;
    }

    constructor() {
        // Deployer gets every role so the full lifecycle can be demonstrated
        // from a single Ganache account.
        _grant(ADMIN_ROLE, msg.sender);
        _grant(MANUFACTURER_ROLE, msg.sender);
        _grant(COURIER_ROLE, msg.sender);
        _grant(LAB_ROLE, msg.sender);
    }

    // ------------------------------------------------------------ role control
    function grantRole(bytes32 role, address account) external onlyRole(ADMIN_ROLE, "Caller is not
ADMIN") {
        _grant(role, account);
    }

    function revokeRole(bytes32 role, address account) external onlyRole(ADMIN_ROLE, "Caller is not
ADMIN") {
        roles[role][account] = false;
        emit RoleRevoked(role, account, msg.sender);
    }

    function hasRole(bytes32 role, address account) external view returns (bool) {
        return roles[role][account];
    }

    function _grant(bytes32 role, address account) private {
        roles[role][account] = true;
        emit RoleGranted(role, account, msg.sender);
    }

    // ------------------------------------------------------------ fingerprint
    /// @notice Commitment to the 10 HPLC peak integers and a secret salt.
    function computeFingerprint(uint256[10] memory peaks, uint256 salt) public pure returns
(bytes32) {
        return keccak256(abi.encodePacked(peaks, salt));
    }

    // ------------------------------------------------------------ lifecycle
    /// @notice MANUFACTURER mints a new batch with its fingerprint commitment.
    function mint(
        string memory drugName,
        string memory batchNumber,
        uint256 quantity,
        uint256 expiryDate,
        bytes32 fingerprint
    ) external onlyRole(MANUFACTURER_ROLE, "Caller is not MANUFACTURER") returns (uint256) {
        require(quantity > 0, "Quantity must be > 0");
        require(expiryDate > block.timestamp, "Expiry must be in the future");
        require(fingerprint != bytes32(0), "Fingerprint required");

        totalBatches++;
        uint256 id = totalBatches;
        batches[id] = Batch({
            tokenId: id,
            drugName: drugName,
            batchNumber: batchNumber,
            quantity: quantity,
            expiryDate: expiryDate,
            fingerprint: fingerprint,
            manufacturer: msg.sender,
            courier: address(0),
            lab: address(0),
            currentHolder: msg.sender,
            status: Status.MINTED,
            mintedAt: block.timestamp
        });
```

```
        _log(id, Status.MINTED);
        emit BatchMinted(id, drugName, batchNumber, quantity, msg.sender, fingerprint);
        return id;
    }

    /// @notice MANUFACTURER hands the batch to a registered COURIER.
    function markInTransit(uint256 tokenId, address courier)
        external
        exists(tokenId)
        onlyRole(MANUFACTURER_ROLE, "Caller is not MANUFACTURER")
    {
        Batch storage b = batches[tokenId];
        require(b.status == Status.MINTED, "Batch must be MINTED");
        require(roles[COURIER_ROLE][courier], "Address is not a COURIER");
        b.courier = courier;
        b.currentHolder = courier;
        b.status = Status.IN_TRANSIT;
        _log(tokenId, Status.IN_TRANSIT);
        emit BatchInTransit(tokenId, courier, block.timestamp);
    }

    /// @notice LAB scans the QR on arrival.
    function confirmReceipt(uint256 tokenId)
        external
        exists(tokenId)
        onlyRole(LAB_ROLE, "Caller is not LAB")
    {
        Batch storage b = batches[tokenId];
        require(b.status == Status.IN_TRANSIT, "Batch must be IN_TRANSIT");
        b.lab = msg.sender;
        b.currentHolder = msg.sender;
        b.status = Status.RECEIVED;
        _log(tokenId, Status.RECEIVED);
        emit BatchReceived(tokenId, msg.sender, block.timestamp);
    }

    /// @notice LAB submits its own HPLC peaks + the salt (Gold Standard channel)
    ///         and the AI anomaly-detector result. Batch becomes VERIFIED only if
    ///         the AI says genuine AND the fingerprint matches the minted one.
    function verifyProof(uint256 tokenId, uint256[10] memory peaks, uint256 salt, bool aiGenuine)
        external
        exists(tokenId)
        onlyRole(LAB_ROLE, "Caller is not LAB")
        returns (bool)
    {
        Batch storage b = batches[tokenId];
        require(b.status == Status.RECEIVED, "Batch must be RECEIVED");

        if (!aiGenuine) {
            b.status = Status.REJECTED;
            _log(tokenId, Status.REJECTED);
            emit BatchRejected(tokenId, msg.sender, "AI anomaly detected", block.timestamp);
            return false;
        }
        if (computeFingerprint(peaks, salt) != b.fingerprint) {
            b.status = Status.REJECTED;
            _log(tokenId, Status.REJECTED);
            emit BatchRejected(tokenId, msg.sender, "Fingerprint mismatch", block.timestamp);
            return false;
        }
        b.status = Status.VERIFIED;
        _log(tokenId, Status.VERIFIED);
        emit BatchVerified(tokenId, msg.sender, block.timestamp);
        return true;
    }

    /// @notice LAB uses the reagent; token is permanently invalidated
    ///         (kill-switch against empty-bottle refill fraud).
    function consumeToken(uint256 tokenId)
        external
```

```
        exists(tokenId)
        onlyRole(LAB_ROLE, "Caller is not LAB")
    {
        Batch storage b = batches[tokenId];
        require(b.status == Status.VERIFIED, "Batch must be VERIFIED");
        b.status = Status.CONSUMED;
        _log(tokenId, Status.CONSUMED);
        emit BatchConsumed(tokenId, msg.sender, block.timestamp);
    }

    // ---------------------------------------------------------------- views
    function getBatch(uint256 tokenId) external view exists(tokenId) returns (Batch memory) {
        return batches[tokenId];
    }

    function getHistory(uint256 tokenId) external view exists(tokenId) returns (HistoryEntry[]
memory) {
        return history[tokenId];
    }

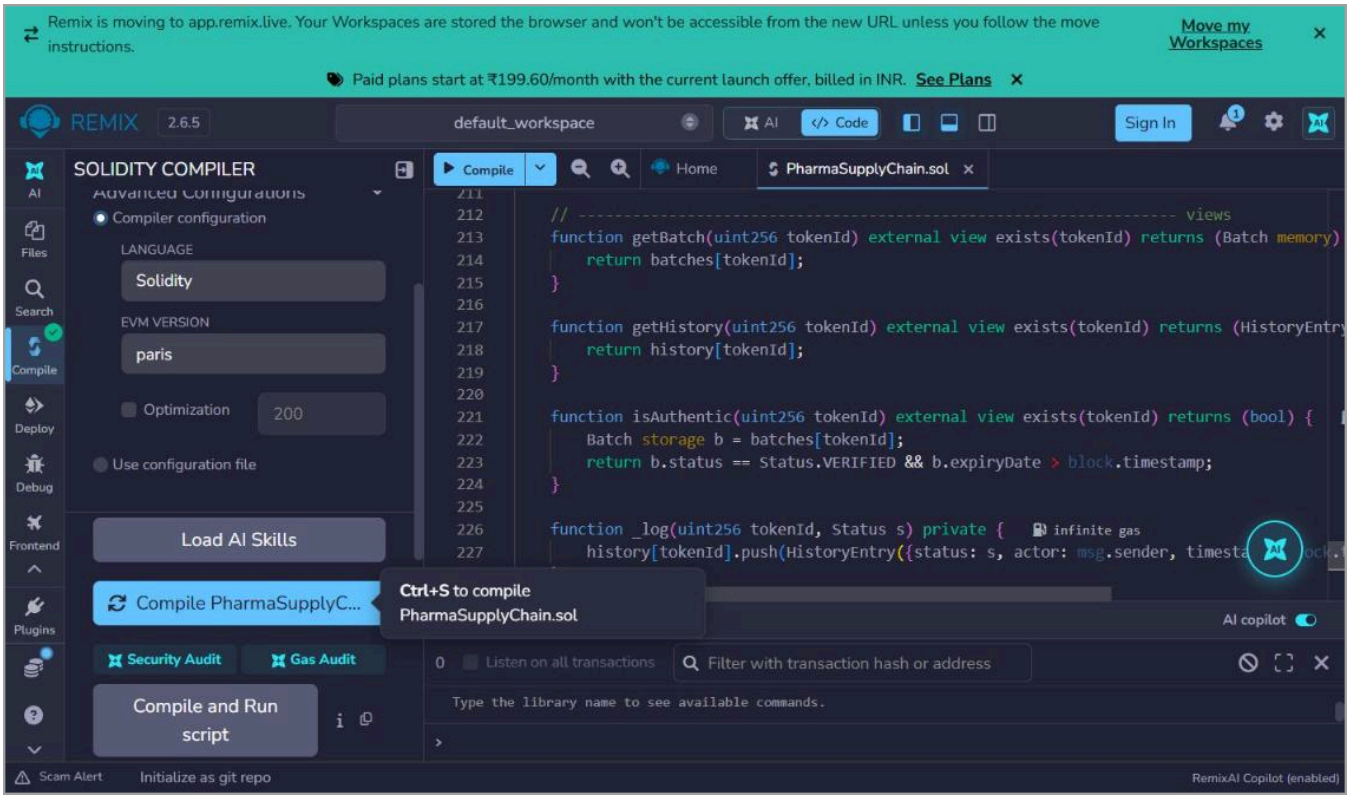
    function isAuthentic(uint256 tokenId) external view exists(tokenId) returns (bool) {
        Batch storage b = batches[tokenId];
        return b.status == Status.VERIFIED && b.expiryDate > block.timestamp;
    }

    function _log(uint256 tokenId, Status s) private {
        history[tokenId].push(HistoryEntry({status: s, actor: msg.sender, timestamp:
block.timestamp}));
    }
}
```

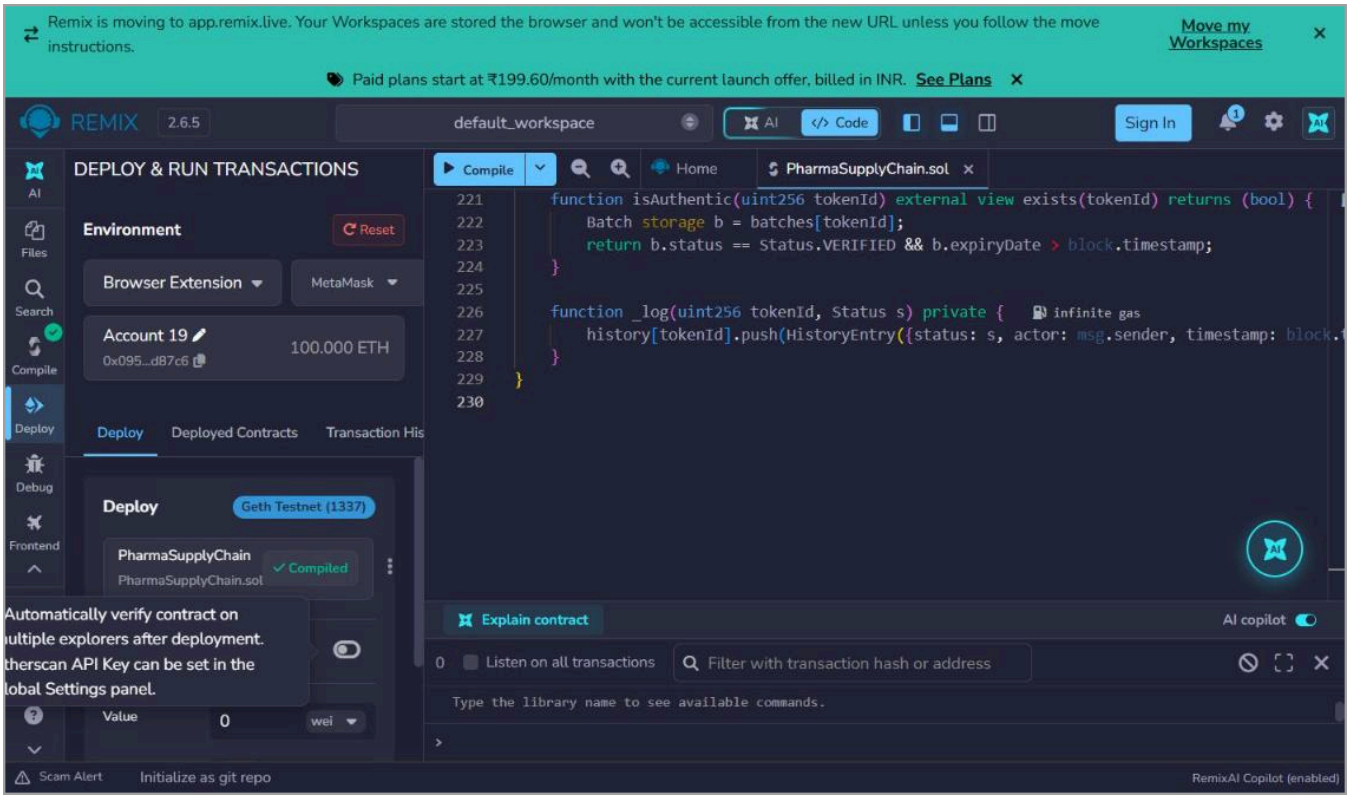
Procedure and Output

The contract was compiled in Remix, deployed through MetaMask (Account 4) to Ganache at `0xfe5EE7B9dD655D2d6eB09D00e33544Ea78b79C0b`, and taken through its whole lifecycle. Batch 1 ended CONSUMED.

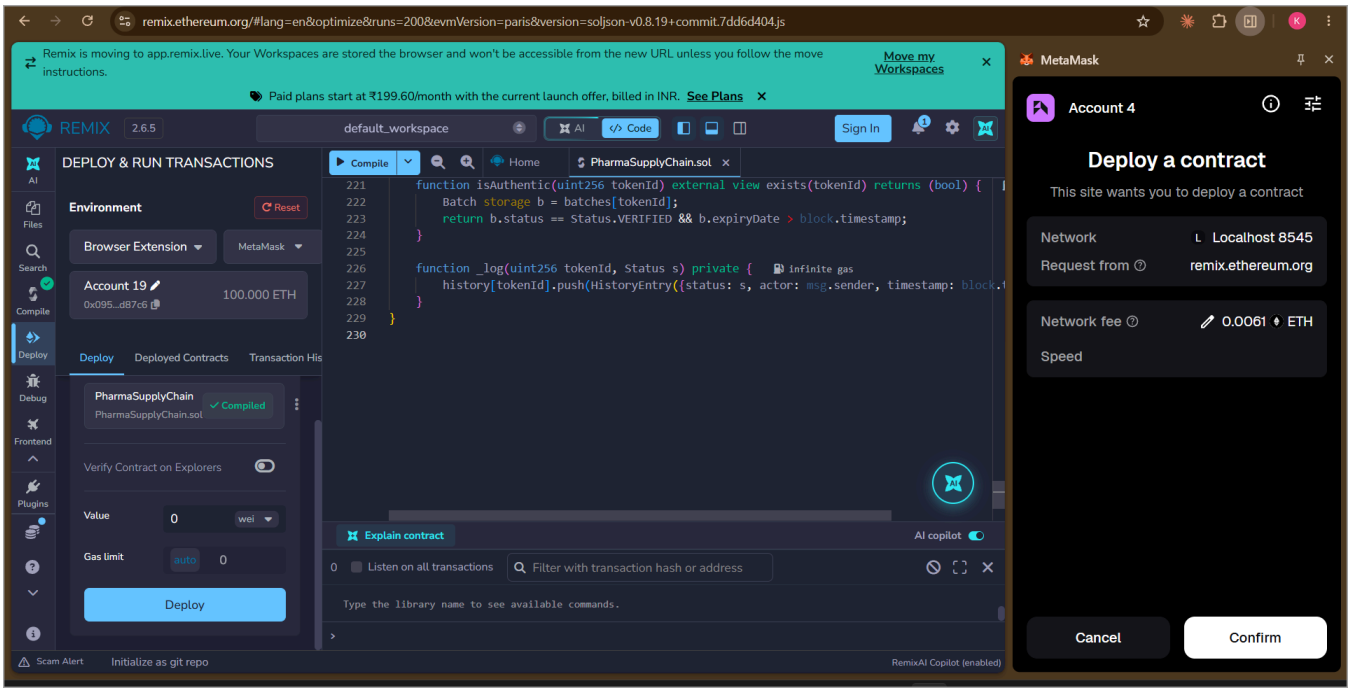
1. Compiled with solc 0.8.19, EVM version paris (green tick on the compiler icon)



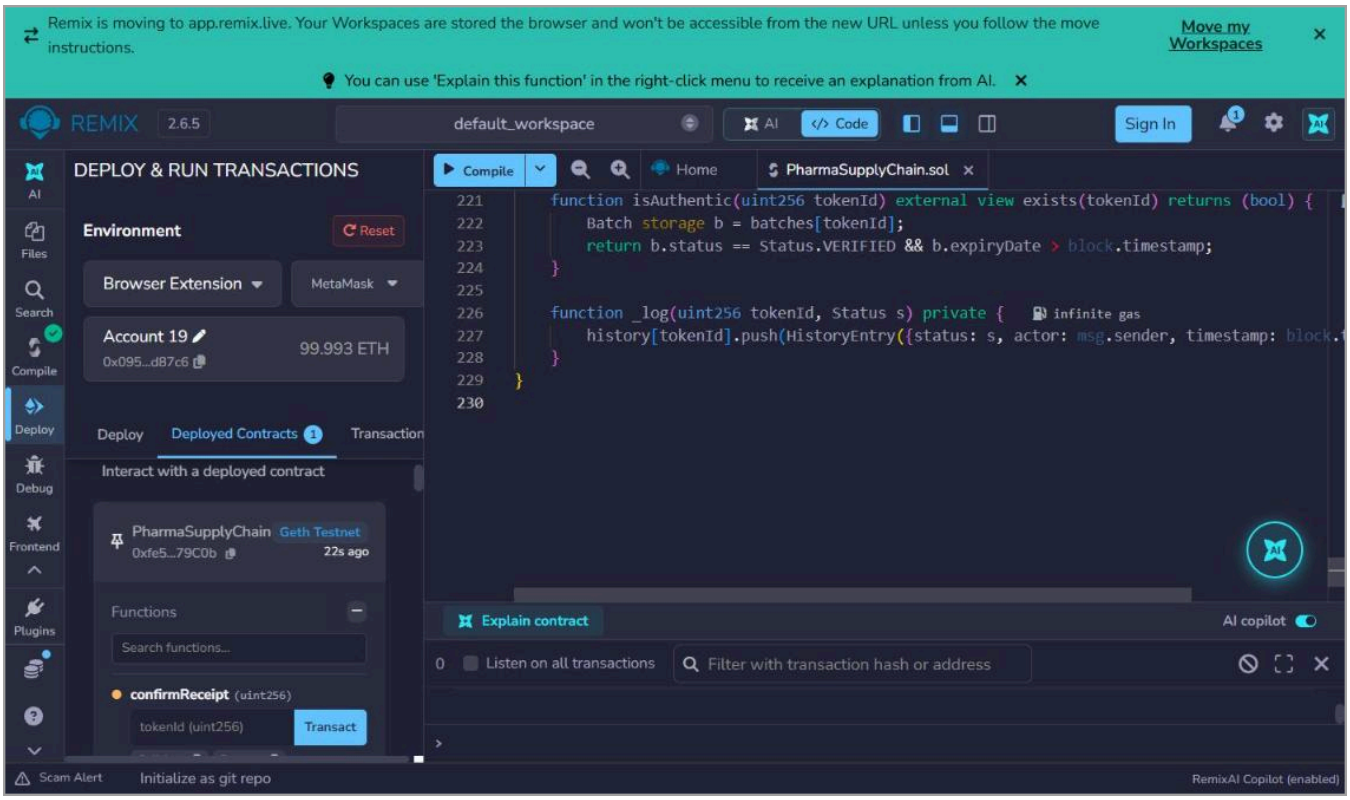
2. Remix connected through Browser Extension → MetaMask: Account 4, 100 ETH, network (1337)



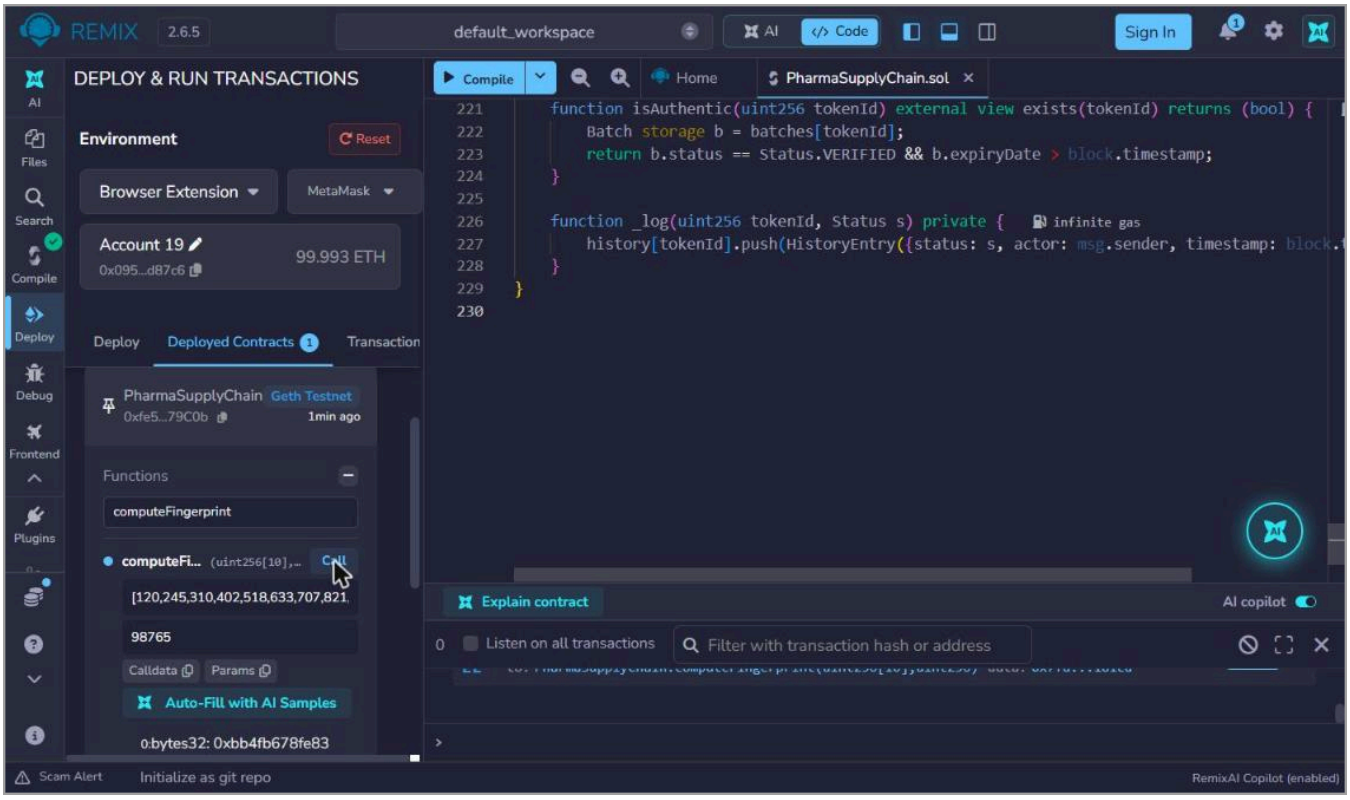
3. MetaMask “Deploy a contract” request on the Ganache network (fee 0.0061 ETH)



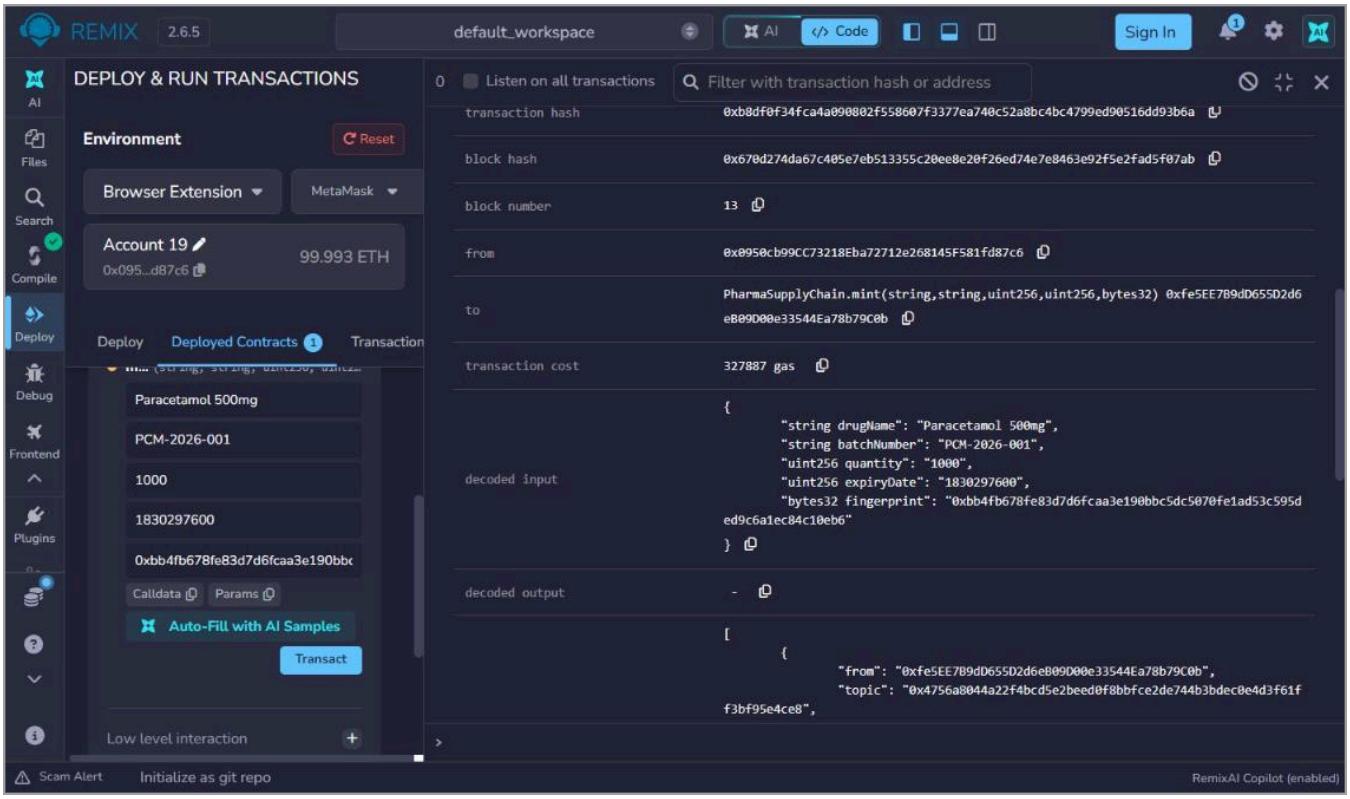
4. PharmaSupplyChain deployed at 0xfe5…79C0b (block 12, 3,025,698 gas)



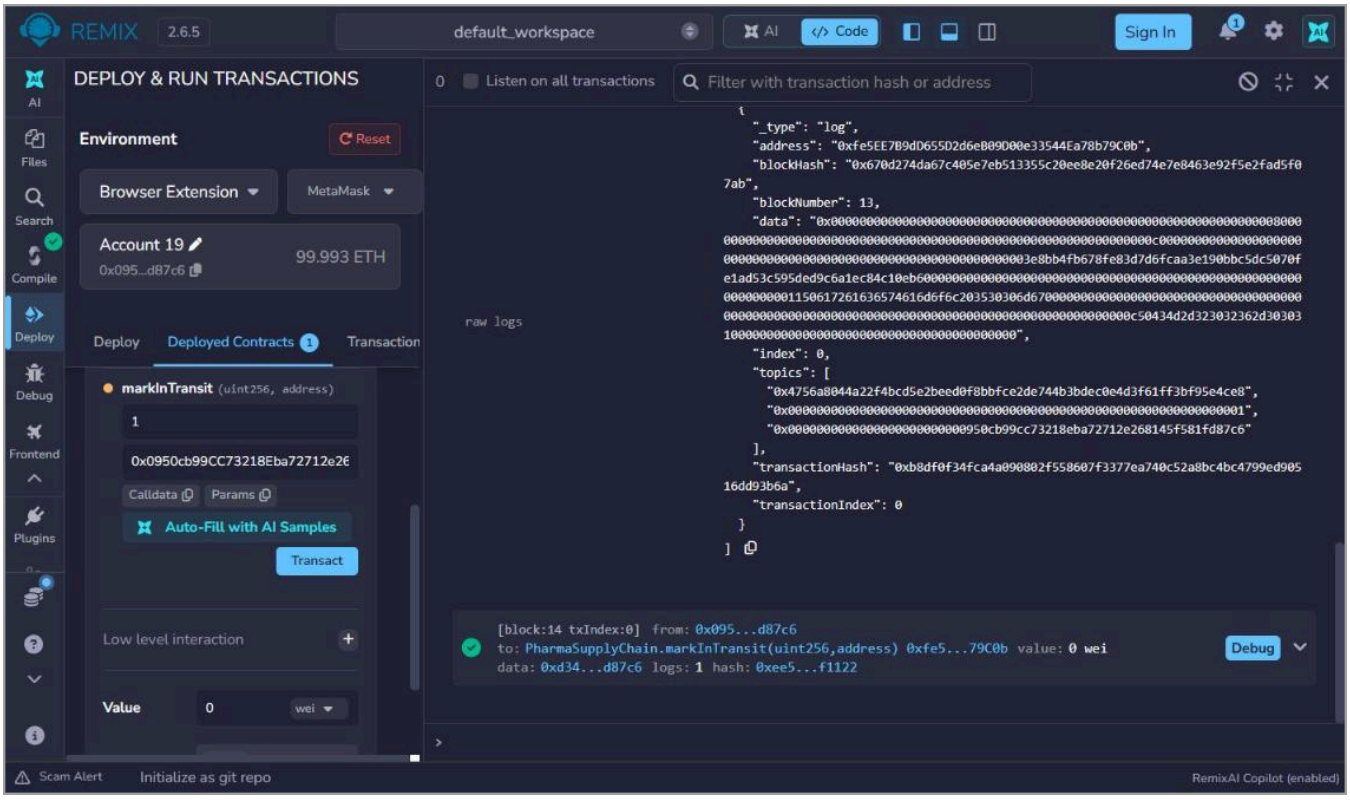
5. computeFingerprint(peaks, 98765) = 0xbb4fb678… (read-only call)



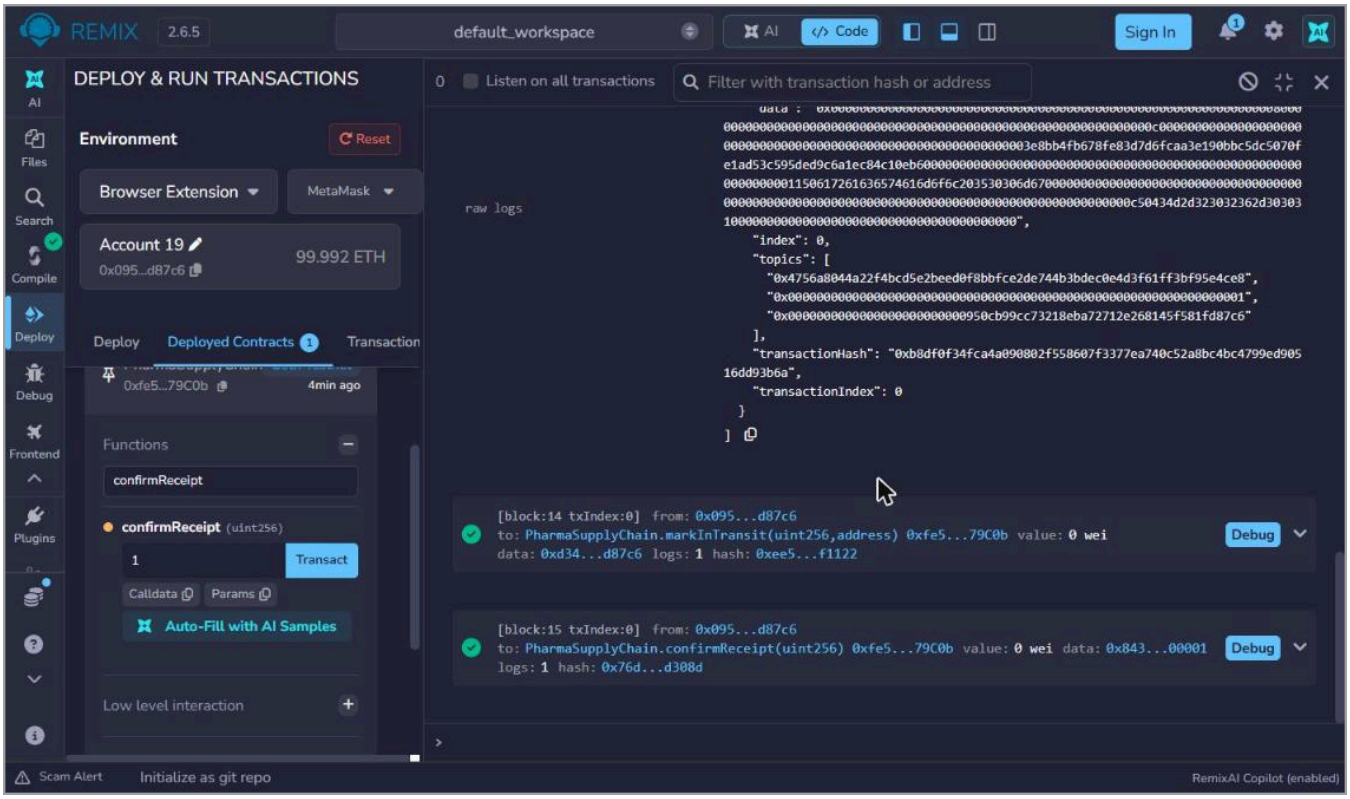
6. mint("Paracetamol 500mg", "PCM-2026-001", 1000, 1830297600, fingerprint): block 13, 327,887 gas, decoded input



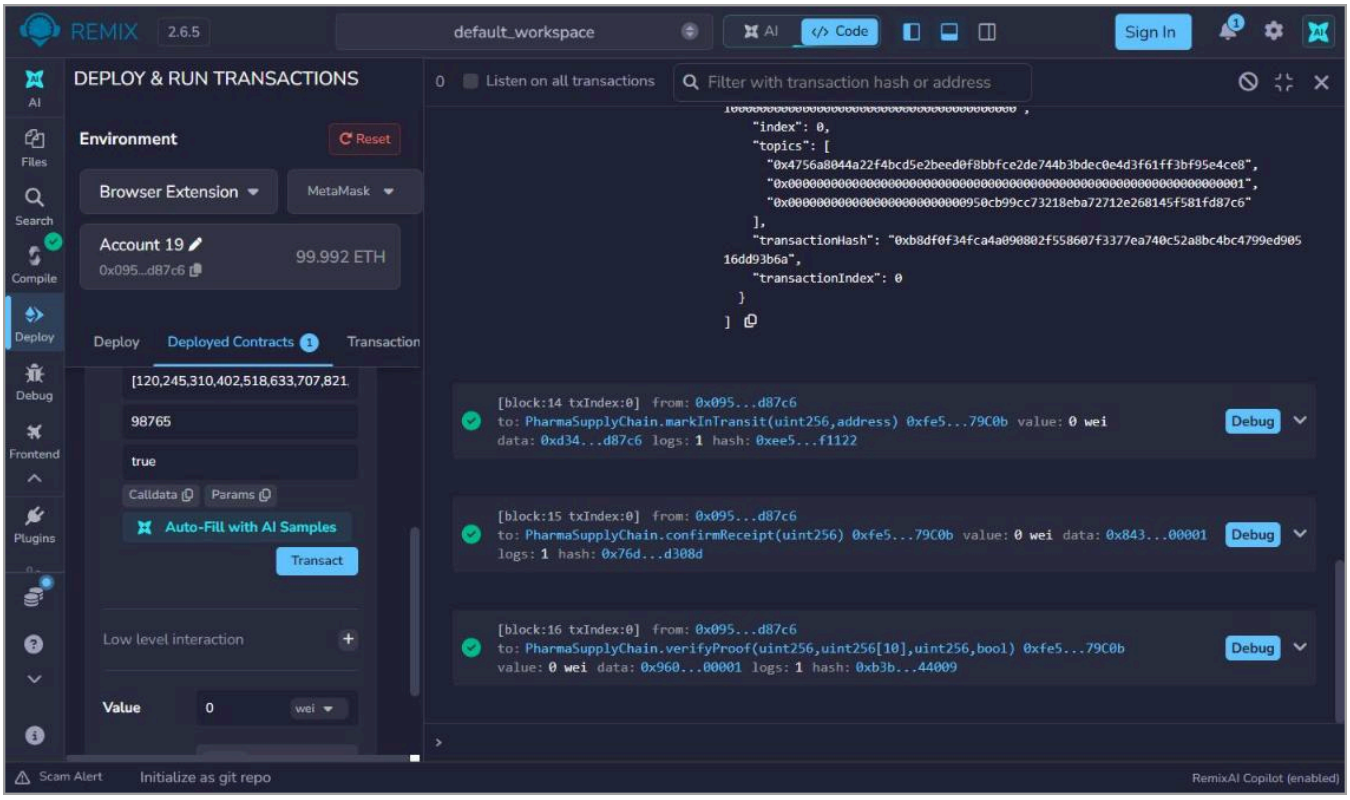
7. markInTransit(1, Account 4): block 14, event BatchInTransit



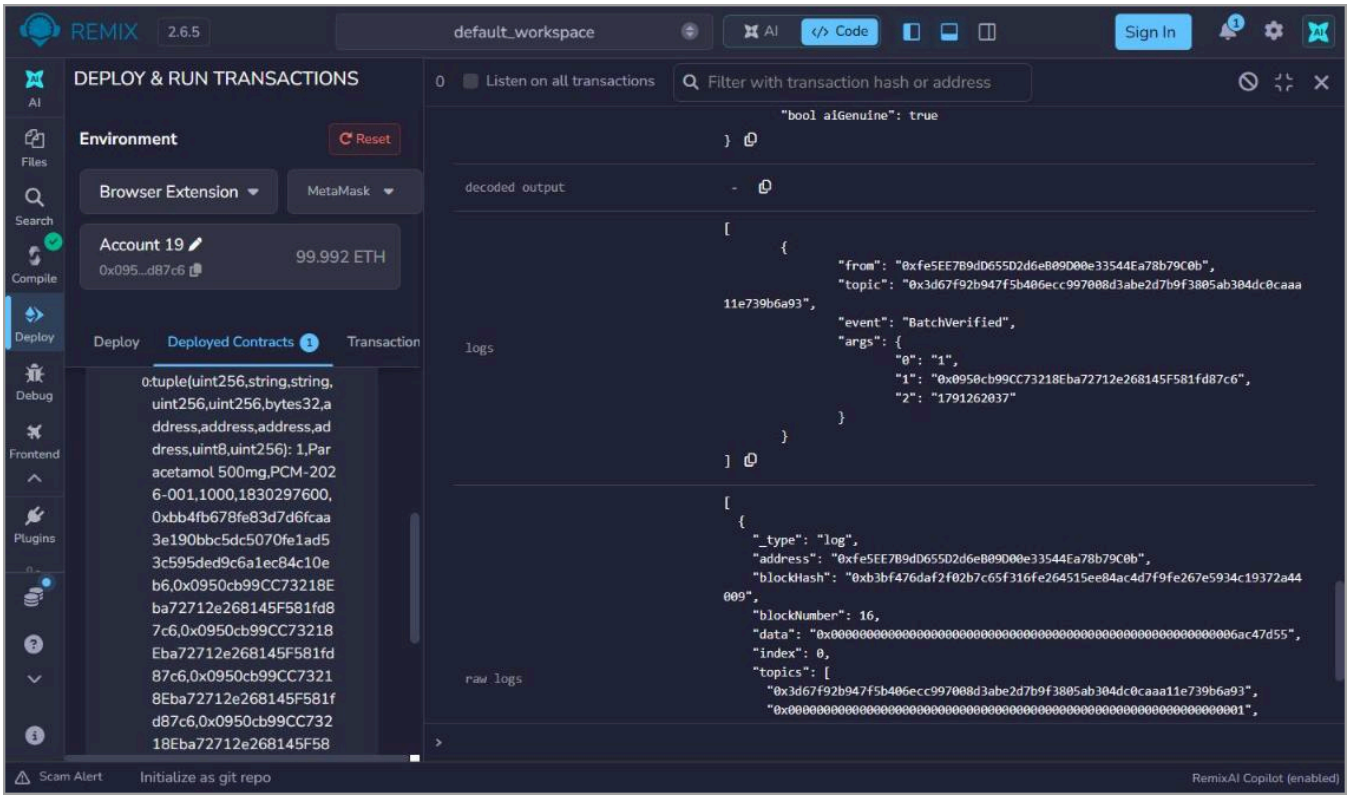
8. confirmReceipt(1): block 15, event BatchReceived



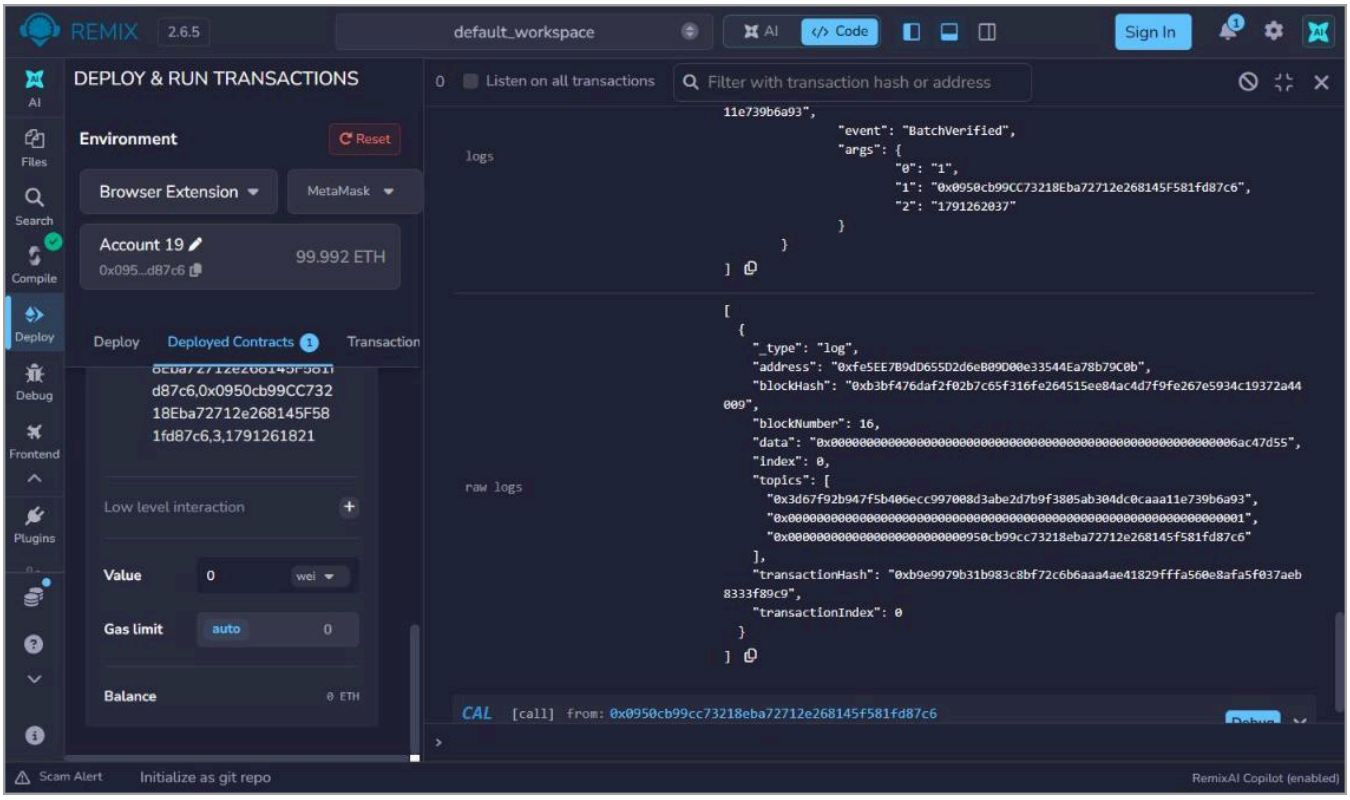
9. verifyProof(1, genuine peaks, 98765, true): block 16, event BatchVerified



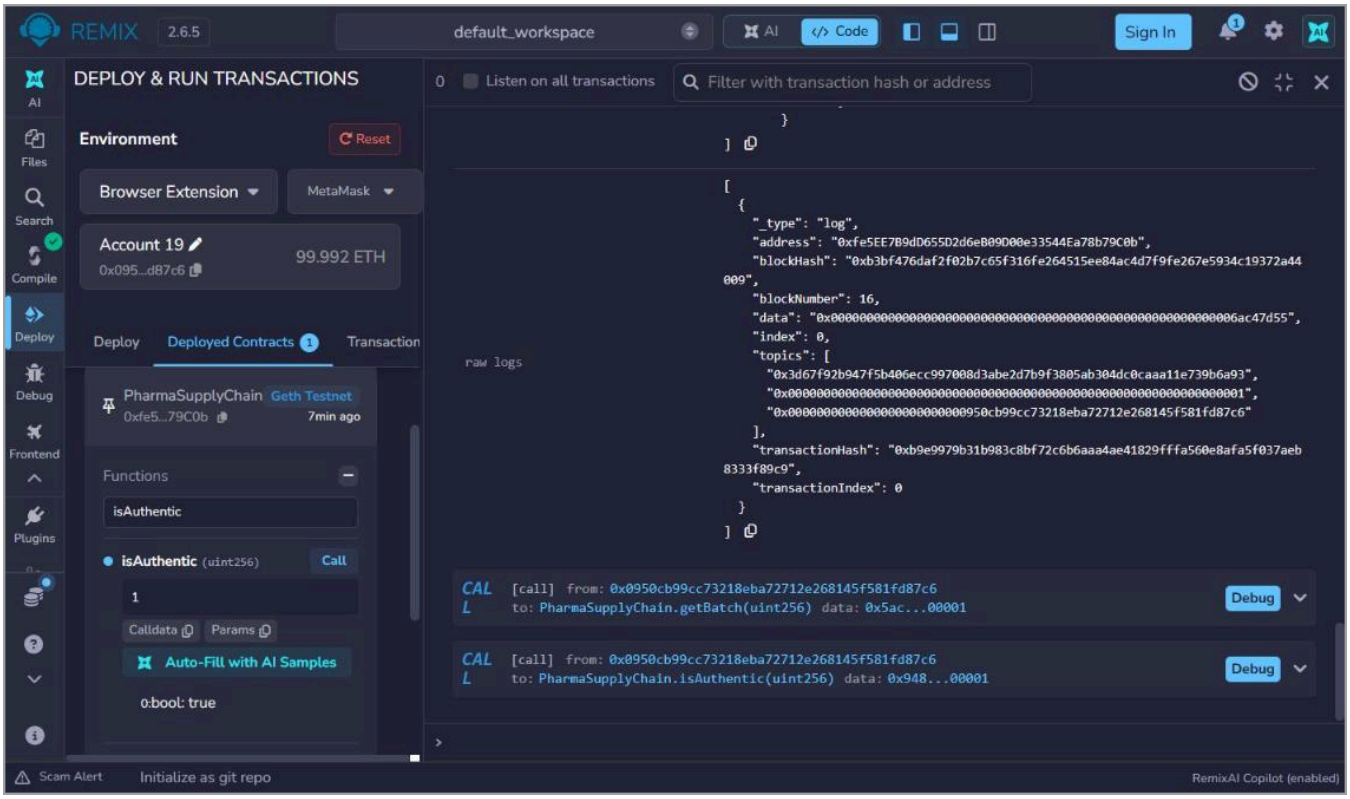
10. getBatch(1): the stored batch record



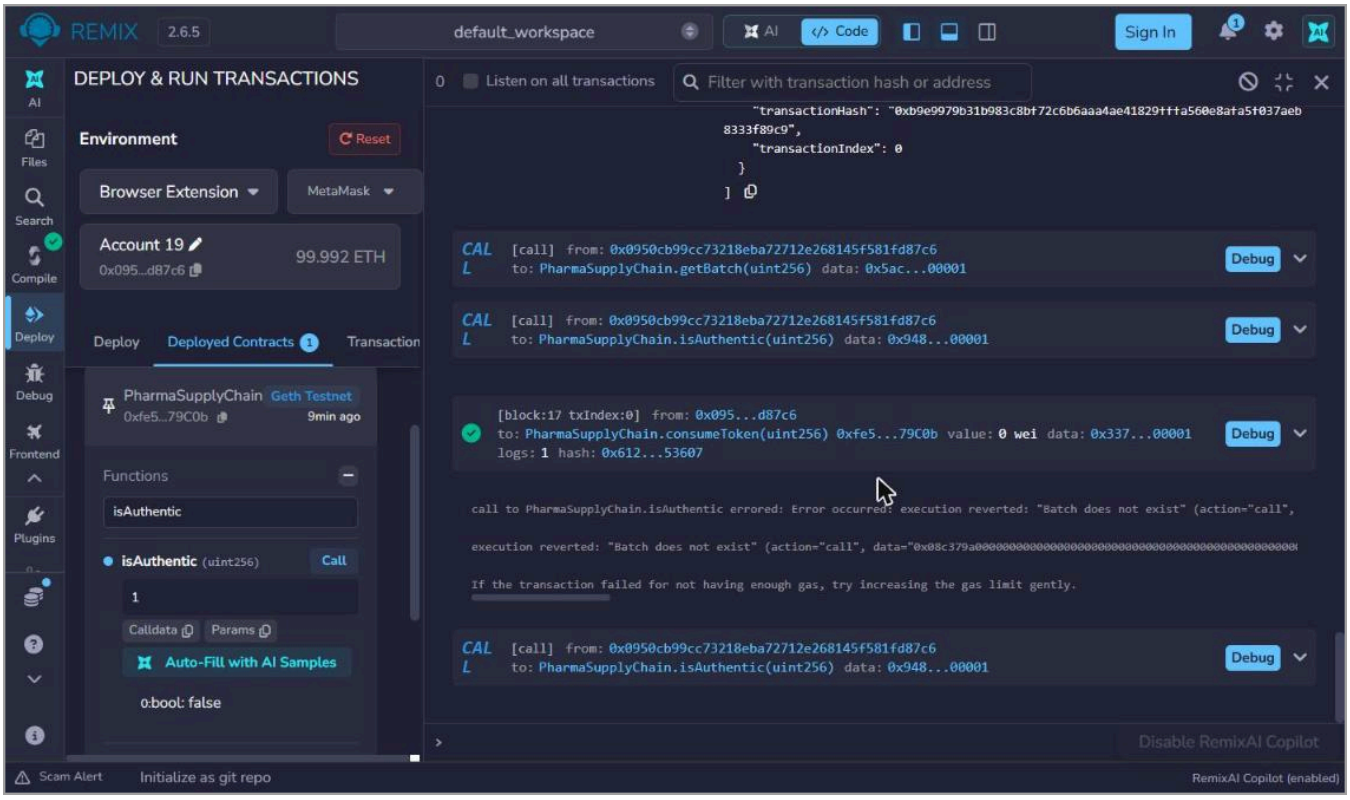
10 (contd.). getBatch(1) ending in status 3 = VERIFIED



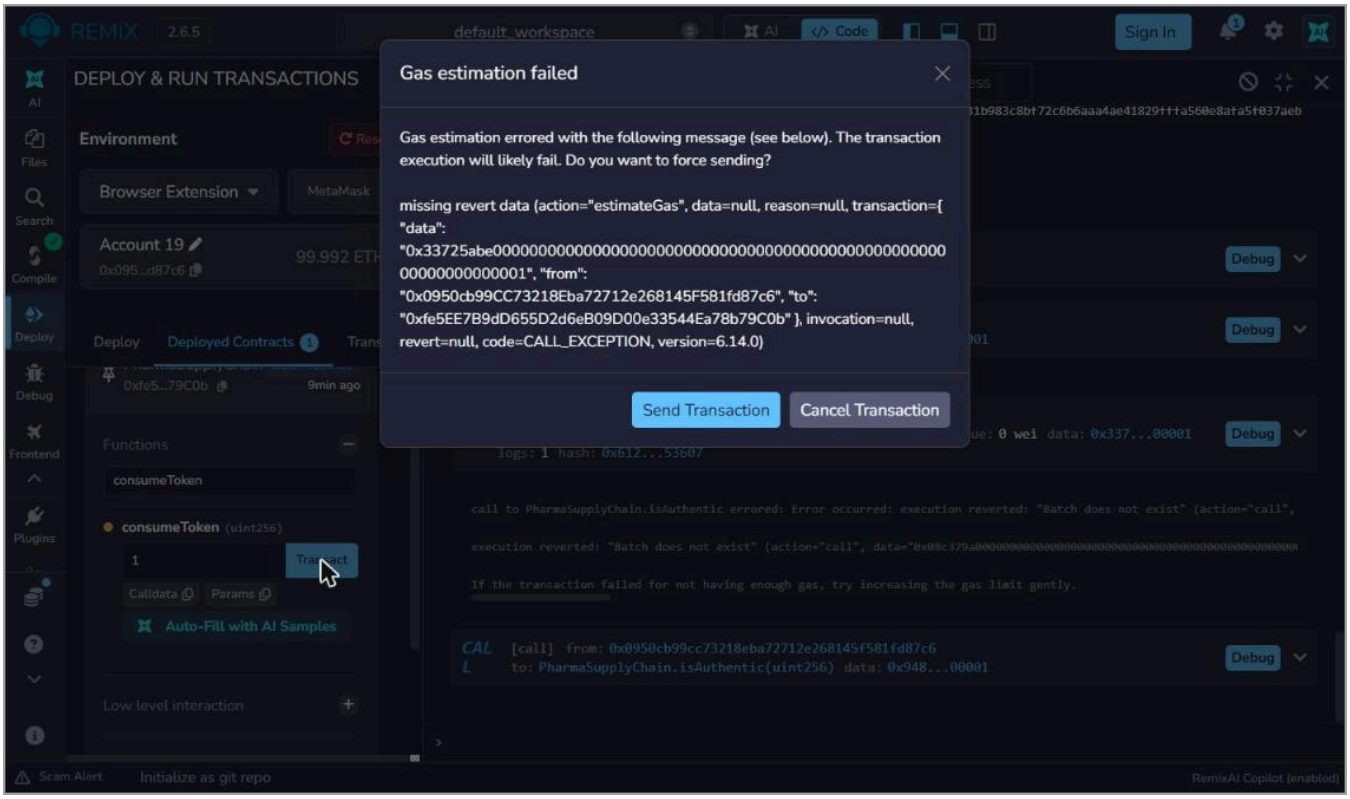
11. isAuthentic(1) = true while VERIFIED



12. consumeToken(1) in block 17, then isAuthentic(1) = false



13. Negative test: consumeToken(1) again fails gas estimation, so it never reaches the chain (revert: “Batch must be VERIFIED”)



Transaction Details on Ganache

All 17 transactions on the Ganache chain succeeded. Blocks 1–10 are the scripted terminal run (ethers.js, Ganache Account 0, contract `0xe78A0F7E598Cc8b0Bb87894B0F60dD2a88d6a8Ab`). Blocks 12–17 are the Remix + MetaMask run (Account 4, contract `0xfe5EE7B9dD655D2d6eB09D00e33544Ea78b79C0b`).

Block	Function	From	Gas used	Tx hash
17	consumeToken(1)	Account 4 (MetaMask)	83,667	`0xb2c53dab91a4b6a558bde2708fe0c0b2dc74487e7f945811655b0d5e02db0c1e`
16	verifyProof(1, peaks, 98765, true)	Account 4 (MetaMask)	94,339	`0xb9e9979b31b983c8bf72c6b6aaa4ae41829fffa560e8afa5f037aeb8333f89c9`
15	confirmReceipt(1)	Account 4 (MetaMask)	106,132	`0x974dfbc7ff74243cc64e4d63d17810297f9657fa64f5b6f48c3d8ed7e783dc21`
14	markInTransit(1, Account 4)	Account 4 (MetaMask)	109,067	`0xe2345f3dba4cf58041d64737ea86183a6f0982e7878cef43c6ac3d160ad1779f`
13	mint("Paracetamol 500mg", "PCM-2026-001", 1000, …)	Account 4 (MetaMask)	327,887	`0xb8df0f34fca4a090802f558607f3377ea740c52a8bc4bc4799ed90516dd93b6a`
12	deploy PharmaSupplyChain (Remix)	Account 4 (MetaMask)	3,025,698	`0xe574bcca004277d85f18f60a60f9ceb095e1ee81396d3b645d4b4b095a59e27a`
11	100 ETH transfer to Account 4	Ganache Account 0	21,000	`0x6efdf2e73e7ed059258d211fe303f245978c6219c0c7e44db46700abec2046ca`
10	verifyProof(2, tampered peaks, 11111, true) → REJECTED	Ganache Account 0	95,307	`0x6bd3c0bd069383a2a59348b1a2bbe20c4d030f305997ee521bc5754b125657a1`
9	confirmReceipt(2)	Ganache Account 0	106,132	`0x316994244e094bf3e5bc0be113a8c08436784852694f10cd14942ab8021764cd`
8	markInTransit(2)	Ganache Account 0	109,067	`0x84a5596881973053f649294c7844f72142bd34fd888293d7a848c093013b425b`
7	mint("Amoxicillin 250mg", "AMX-2026-002", 500, …)	Ganache Account 0	310,787	`0x809920c5dad70116ddb0bd3ef73bfd7296521900f06cb0c9d8b2998cc66c69da`
6	consumeToken(1)	Ganache Account 0	83,667	`0xbc2aea1b0789daf7e5d16e4b1da6a1e165f7adf14ed2bc3bb4247ce4d518f6ef`
5	verifyProof(1, peaks, 98765, true)	Ganache Account 0	94,339	`0xfc9387d21ed58bc88a052b79b2ee1a1bb724dd6d31e936d379e3243b9bc681a1`
4	confirmReceipt(1)	Ganache Account 0	106,132	`0x18d5004ace58541924e0ce5b70037c972b69e8ec24bac6bd9677aa6bbff7c126`
3	markInTransit(1)	Ganache Account 0	109,067	`0xa45e2f2987311a21056bdc18ea3a97e1329a15cd929225ab8b807209629b157e`
2	mint("Paracetamol 500mg", "PCM-2026-001", 1000, …)	Ganache Account 0	327,887	`0x4fb307fc636a5310ffc6f8bb86d684dd47977c786c37e6afdd7f2c3a1052ed38`
1	deploy PharmaSupplyChain (terminal)	Ganache Account 0	3,025,698	`0x08071b339335d05db7901fb1a9868b4f8a4942ff6a61103d758a16228047c7f2`

Final balances: Ganache Account 0 = 899.9957 ETH (100 ETH sent to MetaMask plus gas), Account 1 = 1000 ETH, MetaMask Account 4 = 99.9925 ETH.

Results of the terminal run (blocks 1–10):

- Batch 1 (Paracetamol): fingerprint `0xbb4fb678fe83d7d6fcaa3e190bbc5dc5070fe1ad53c595ded9c6a1ec84c10eb6`. It reached VERIFIED with `isAuthentic` = true, then CONSUMED with `isAuthentic` = false.
- Tamper detection: batch 2 was minted with salt 11111. At verification one peak was changed (1004 → 999), so the contract emitted `BatchRejected("Fingerprint mismatch")` and the status is REJECTED.
- Re-consuming batch 1 reverts with `"Batch must be VERIFIED"`.
- Access control: `mint` from Ganache Account 1 (no roles) reverts with `"Caller is not MANUFACTURER"`.

The short "hash" in Remix's collapsed log line is the block hash. The transaction hashes above are Ganache's, and they match the `transactionHash` in Remix's expanded logs.

Outcome

- Understood the steps for linking the **MetaMask** wallet with **Remix IDE** and performing transactions.
- Deployed `PharmaSupplyChain` and ran transactions from Remix through a MetaMask account on **Ganache**: 1 deploy and 5 lifecycle calls, each confirmed in MetaMask.
- Verified every transaction, block, gas figure and balance on the Ganache chain (blocks 0–17, all successful).

Conclusion

Ganache provides a simple, powerful local Ethereum blockchain. Integrated with MetaMask and Remix, it let us deploy, test and debug the pharmaceutical supply-chain contract at no cost before moving to a public network. The contract enforced the BioToken lifecycle on-chain. A genuine batch went MINTED → IN_TRANSIT → RECEIVED → VERIFIED → CONSUMED. A batch with tampered HPLC peaks was REJECTED. A consumed token could not be reused, and a caller without the MANUFACTURER role could not mint.